

NOIP 模拟赛讲评

SnowTrace

2025.10.27

废话

好像出的有点难。

题意

定义长度为 n 的序列 $\{b\}$ 的权值为 $\frac{\text{mex}(b)}{n}$ 。

给定一个长度为 n 的序列 a ，你需要求出 a 的所有子区间（即所有 $1 \leq l \leq r \leq n$ ， $\{a_l, a_{l+1}, \dots, a_r\}$ ）中权值最大的子区间的数量。

分析

- 能取到最大值的区间有什么性质？

分析

- 能取到最大值的区间有什么性质？
- 容易发现，如果序列中不存在 0， mex 就永远是 0，从而此时最大值是 0，所有区间都能取到，答案是 $\frac{n \times (n+1)}{2}$

分析

- 能取到最大值的区间有什么性质？
- 容易发现，如果序列中不存在 0， mex 就永远是 0，从而此时最大值是 0，所有区间都能取到，答案是 $\frac{n \times (n+1)}{2}$
- 如果存在 0，考虑 0 这一个数所对应的子区间，这个子区间权值是 1，也容易发现实际上一个序列的权值不可能超过 1，因此此时最大值是 1

分析

- 能取到最大值的区间有什么性质?
- 容易发现, 如果序列中不存在 0 , mex 就永远是 0 , 从而此时最大值是 0 , 所有区间都能取到, 答案是 $\frac{n \times (n+1)}{2}$
- 如果存在 0 , 考虑 0 这一个数所对应的子区间, 这个子区间权值是 1 , 也容易发现实际上一个序列的权值不可能超过 1 , 因此此时最大值是 1
- 实际上这样的区间需要满足其 mex 等于其长度, 也就是 $0, 1, \dots, m-1$ 必须都出现

算法思路

- 直接暴力枚举每一个区间计算其 mex, 复杂度 $O(n^2)$

算法思路

- 直接暴力枚举每一个区间计算其 mex, 复杂度 $O(n^2)$
- 可以利用一些手段优化, 比如笛卡尔树分治, 分治, 线段树等, 可以做到 $O(n \log n)$

算法思路

- 直接暴力枚举每一个区间计算其 mex, 复杂度 $O(n^2)$
- 可以利用一些手段优化, 比如笛卡尔树分治, 分治, 线段树等, 可以做到 $O(n \log n)$
- 题目的限制很强, 尝试推出更多性质优化到线性

题意

初始有一个整数 x ，你需要对其进行 n 轮操作，第 i 轮操作会做如下两件事情之一：

1. 令 x 变为 $\max(0, x - a_i)$
2. 令 x 变为 $x + b_i$

你需要对每个 k 求出恰好做 k 次第二种操作和 $n - k$ 次第一种操作最终 x 可以达到的最大值。

分析

- 如何快速刻画一组决策最终得到的 x 的值?

分析

- 如何快速刻画一组决策最终得到的 x 的值?
- 尝试钦定最后一个归零的位置, 此时的策略是什么?

分析

- 如何快速刻画一组决策最终得到的 x 的值?
- 尝试钦定最后一个归零的位置, 此时的策略是什么?
- 按照后缀的 $a_i + b_i$ 排序, 取前 k 大。

算法思路

- 暴力枚举每个位置，求出每个位置后缀前 k 大值的和，复杂度 $O(n^2)$ 。

算法思路

- 暴力枚举每个位置，求出每个位置后缀前 k 大值的和，复杂度 $O(n^2)$ 。
- 乱搞？

算法思路

- 暴力枚举每个位置，求出每个位置后缀前 k 大值的和，复杂度 $O(n^2)$ 。
- 乱搞？
- 尝试分析整体过程。

算法思路

- 暴力枚举每个位置，求出每个位置后缀前 k 大值的和，复杂度 $O(n^2)$ 。
- 乱搞？
- 尝试分析整体过程。
- 决策在某种意义上有单调性，整体二分后用对顶堆动态维护前 k 大和，复杂度 $O(n \log^2 n)$ 。

算法思路

- 暴力枚举每个位置，求出每个位置后缀前 k 大值的和，复杂度 $O(n^2)$ 。
- 乱搞？
- 尝试分析整体过程。
- 决策在某种意义上有单调性，整体二分后用对顶堆动态维护前 k 大和，复杂度 $O(n \log^2 n)$ 。
- $1log$ 做法？

题意

求出满足以下条件的 01 序列个数：

1. a 中恰好有 n 个 0。
2. a 中恰好有 m 个 1。
3. a 不存在一个子区间使得其中 1 的数量和 0 的数量差（的绝对值） $> k$ 。

现在给定 n, m ，你需要对所有 $1 \leq k \leq n + m$ 求出答案，对 998244353 取模。

分析 + 思路

- 暴力枚举所有序列，复杂度很高。

分析 + 思路

- 暴力枚举所有序列，复杂度很高。
- 尝试用一个 dp 解决，复杂度仍然很高。

分析 + 思路

- 暴力枚举所有序列，复杂度很高。
- 尝试用一个 dp 解决，复杂度仍然很高。
- 尝试转化，将 01 序列转化为格路。

分析 + 思路

- 暴力枚举所有序列，复杂度很高。
- 尝试用一个 dp 解决，复杂度仍然很高。
- 尝试转化，将 01 序列转化为格路。
- 尝试使用反射容斥解决。

分析 + 思路

- 暴力枚举所有序列，复杂度很高。
- 尝试用一个 dp 解决，复杂度仍然很高。
- 尝试转化，将 01 序列转化为格路。
- 尝试使用反射容斥解决。
- 对于一个 k ，做一次反射容斥的复杂度是 $O(\frac{n}{k})$ ，而且需要做 k 次反射容斥，因此此时算一个 k 的复杂度恰好是 $O(n)$ ，总复杂度 $O(n^2)$ 。

分析 + 思路

- 暴力枚举所有序列，复杂度很高。
- 尝试用一个 dp 解决，复杂度仍然很高。
- 尝试转化，将 01 序列转化为格路。
- 尝试使用反射容斥解决。
- 对于一个 k ，做一次反射容斥的复杂度是 $O(\frac{n}{k})$ ，而且需要做 k 次反射容斥，因此此时算一个 k 的复杂度恰好是 $O(n)$ ，总复杂度 $O(n^2)$ 。
- 如何优化？

分析 + 思路

- 暴力枚举所有序列，复杂度很高。
- 尝试用一个 dp 解决，复杂度仍然很高。
- 尝试转化，将 01 序列转化为格路。
- 尝试使用反射容斥解决。
- 对于一个 k ，做一次反射容斥的复杂度是 $O(\frac{n}{k})$ ，而且需要做 k 次反射容斥，因此此时算一个 k 的复杂度恰好是 $O(n)$ ，总复杂度 $O(n^2)$ 。
- 如何优化？
- 观察反射容斥整体代码，可以实现 k 次反射容斥并行运行，时间复杂度 $O(n \log n)$ 。

分析 + 思路

- 暴力枚举所有序列，复杂度很高。
- 尝试用一个 dp 解决，复杂度仍然很高。
- 尝试转化，将 01 序列转化为格路。
- 尝试使用反射容斥解决。
- 对于一个 k ，做一次反射容斥的复杂度是 $O(\frac{n}{k})$ ，而且需要做 k 次反射容斥，因此此时算一个 k 的复杂度恰好是 $O(n)$ ，总复杂度 $O(n^2)$ 。
- 如何优化？
- 观察反射容斥整体代码，可以实现 k 次反射容斥并行运行，时间复杂度 $O(n \log n)$ 。
- ps: 推式子也能得到相同结果。

题意

有 n 个波特，初始第一个波特知道秘密，秘密在波特相遇时传递，波特只能从同一时刻开始一起朝某个固定方向走，你需要决定每个波特的行走方向使得知道秘密的波特最多。

分析

■ 怎么写暴力？

分析

- 怎么写暴力？
- 初步分析可以发现一个波特最多朝两个方向才有可能和别的波特相遇，否则它自己越走越远。

分析

- 怎么写暴力？
- 初步分析可以发现一个波特最多朝两个方向才有可能和别的波特相遇，否则它自己越走越远。
- 无法求出最大值的困难点在哪？

分析

- 怎么写暴力？
- 初步分析可以发现一个波特最多朝两个方向才有可能和别的波特相遇，否则它自己越走越远。
- 无法求出最大值的困难点在哪？
- 尝试用更好的方式刻画相遇。

分析

- 怎么写暴力？
- 初步分析可以发现一个波特最多朝两个方向才有可能和别的波特相遇，否则它自己越走越远。
- 无法求出最大值的困难点在哪？
- 尝试用更好的方式刻画相遇。
- 关键结论，在一号波特方向固定的情况下，每个波特只有唯一一种方向的选择使得它有可能碰到别的波特。

分析

- 怎么写暴力？
- 初步分析可以发现一个波特最多朝两个方向才有可能和别的波特相遇，否则它自己越走越远。
- 无法求出最大值的困难点在哪？
- 尝试用更好的方式刻画相遇。
- 关键结论，在一号波特方向固定的情况下，每个波特只有唯一一种方向的选择使得它有可能碰到别的波特。
- 于是只需要维护相遇的过程。

算法思路

- 对每两个波特之间判定一次，执行 n 轮，类似 *bellman - ford*，复杂度 $O(n^3)$ ，优点是很好写。

算法思路

- 对每两个波特之间判定一次，执行 n 轮，类似 *bellman - ford*，复杂度 $O(n^3)$ ，优点是很好写。
- 这个结构可以类似 *dijkstra* 一样，储存每个波特最早激活时间，每次拿出最小的进行松弛，复杂度 $O(n^2 \log n)$ 。

算法思路

- 对每两个波特之间判定一次，执行 n 轮，类似 *bellman - ford*，复杂度 $O(n^3)$ ，优点是很好写。
- 这个结构可以类似 *dijkstra* 一样，储存每个波特最早激活时间，每次拿出最小的进行松弛，复杂度 $O(n^2 \log n)$ 。
- 实际上只会增广固定的两条斜线和水平竖直线上的点。

算法思路

- 对每两个波特之间判定一次，执行 n 轮，类似 *bellman - ford*，复杂度 $O(n^3)$ ，优点是很好写。
- 这个结构可以类似 *dijkstra* 一样，储存每个波特最早激活时间，每次拿出最小的进行松弛，复杂度 $O(n^2 \log n)$ 。
- 实际上只会增广固定的两条斜线和水平竖直线上的点。
- 用数据结构维护所有的线上的点。

算法思路

- 对每两个波特之间判定一次，执行 n 轮，类似 *bellman - ford*，复杂度 $O(n^3)$ ，优点是很好写。
- 这个结构可以类似 *dijkstra* 一样，储存每个波特最早激活时间，每次拿出最小的进行松弛，复杂度 $O(n^2 \log n)$ 。
- 实际上只会增广固定的两条斜线和水平竖直线上的点。
- 用数据结构维护所有的线上的点。
- 想办法编一个好写一点的做法。

算法思路

- 对每两个波特之间判定一次，执行 n 轮，类似 *bellman - ford*，复杂度 $O(n^3)$ ，优点是很好写。
- 这个结构可以类似 *dijkstra* 一样，储存每个波特最早激活时间，每次拿出最小的进行松弛，复杂度 $O(n^2 \log n)$ 。
- 实际上只会增广固定的两条斜线和水平竖直线上的点。
- 用数据结构维护所有的线上的点。
- 想办法编一个好写一点的做法。
- 用 *set* 维护松弛过程，每次贡献至斜线上最近能贡献的点，再由这个点将信息传给要松弛的斜线上的下一个点。

算法思路

- 对每两个波特之间判定一次，执行 n 轮，类似 *bellman - ford*，复杂度 $O(n^3)$ ，优点是很好写。
- 这个结构可以类似 *dijkstra* 一样，储存每个波特最早激活时间，每次拿出最小的进行松弛，复杂度 $O(n^2 \log n)$ 。
- 实际上只会增广固定的两条斜线和水平竖直线上的点。
- 用数据结构维护所有的线上的点。
- 想办法编一个好写一点的做法。
- 用 *set* 维护松弛过程，每次贡献至斜线上最近能贡献的点，再由这个点将信息传给要松弛的斜线上的下一个点。
- 复杂度 $O(n \log n)$ ，常数巨大。

算法思路

- 对每两个波特之间判定一次，执行 n 轮，类似 *bellman - ford*，复杂度 $O(n^3)$ ，优点是很好写。
- 这个结构可以类似 *dijkstra* 一样，储存每个波特最早激活时间，每次拿出最小的进行松弛，复杂度 $O(n^2 \log n)$ 。
- 实际上只会增广固定的两条斜线和水平竖直线上的点。
- 用数据结构维护所有的线上的点。
- 想办法编一个好写一点的做法。
- 用 *set* 维护松弛过程，每次贡献至斜线上最近能贡献的点，再由这个点将信息传给要松弛的斜线上的下一个点。
- 复杂度 $O(n \log n)$ ，常数巨大。
- 本题性质很强，是否存在更好写的做法？

A

如何对子区间计数？

$mex = len$

$max = len - 1$

枚举最大值，长度确定。

li, ri 表示左边第一个比某个数大的数

$\min(i - li, ri - i)$ 相当于启发式合并。

检查 $n \log n$ 个区间

分治：枚举最大值位置。

扫描线：定义序列权值是 $len - sum_i[i \text{ 和 } i + 1 \text{ 都出现的个数}]$ ，只需要计算存在一个 0 的所有的权值 = 1 区间。

线段树维护。

$max = len - 1$

0 恰好出现一次

用 0 划分若干段，每一段单独

pre_i 前缀最大值

$pre_i - i$ 左边序列应该取到的长度

总变化量是 $O(n)$ 的。

给每个数赋权，定义集合的权值是所有数权值的 xor

$xor - hashing$

B

T2

考察折线的最低点

最低点一定是 0。

相当于，取一段后缀，这段后缀里面直接加/直接减。

 $s - mn$ mn 折线最低点 $s - \min_i \{a_i\}$ 任选 $a_i, \min\{s - a_i\}$

钦定最小值/最后归零的点。

感性理解

选 $k + 1$ 点的位置选 k 点的位置

靠前

 $i + 1$ 选定 pos_i 为后缀

决策单调性。

已经知道决策点单调递增。

做双指针需要保证 $i + 1$ 一定比 i 更优 mid ，最优决策点是 p' $< mid$ 决策点 $\leq p'$ $> mid$ 决策点 $\geq p'$ $n \log n$ 次计算 $cntsum_i[a_i! = b_i]$

祝大家
CSP RP++